Tutoriel Flutter

Signer & Publier un APK sur GitHub

Créer une Keystore · Générer un APK signé · Créer une Release GitHub

Prérequis

- Flutter installé et configuré (`flutter doctor` sans erreur critique)
- Android Studio installé (fournit le JDK et `keytool`)
- Un projet Flutter existant et fonctionnel
- 📋 *Repo GitHub : votre projet Flutter doit déjà être lié à un repo GitHub avec au moins un commit pushé. Si ce n'est pas encore fait, créez votre repo sur github.com et poussez votre projet avant de continuer.*

Si keytool n'est pas disponible

Si la commande keytool renvoie command not found, deux options :

— **Via Android Studio (recommandé)** Ajoutez ce chemin à votre ~/.zshrc :

```
export PATH="$PATH:/Applications/Android
Studio.app/Contents/jbr/Contents/Home/bin"
```

Puis rechargez le terminal :

```
source ~/.zshrc
```

Étape 1 — Vérifier que keytool est disponible

Ouvrez un terminal et tapez :

Vous devez voir la liste des commandes disponibles de keytool. Cela confirme que l'outil est bien accessible.

```
● renelsmac@Mac-Renel saws_app % keytool --help
  Key and Certificate Management Tool

  Commands:

   -certreq            Generates a certificate request
   -changealias        Changes an entry's alias
   -delete             Deletes an entry
   -exportcert         Exports certificate
   -genkeypair         Generates a key pair
   -genseckey          Generates a secret key
   -gencert            Generates certificate from a certificate request
   -importcert         Imports a certificate or a certificate chain
   -importpass         Imports a password
   -importkeystore     Imports one or all entries from another keystore
   -keypasswd          Changes the key password of an entry
   -list               Lists entries in a keystore
   -printcert          Prints the content of a certificate
   -printcertreq       Prints the content of a certificate request
   -printcrl           Prints the content of a CRL file
   -storepasswd        Changes the store password of a keystore
   -showinfo           Displays security-related information
```

Étape 2 — Créer la Keystore

Copiez et collez cette commande dans votre terminal (les \ permettent d'écrire sur plusieurs lignes, c'est une seule commande) :

```
keytool -genkeypair \
  -alias upload \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -keystore ~/upload-keystore.jks \
  -storetype JKS
```

Détail des paramètres :

Paramètre	Rôle
`-alias upload`	Nom de la clé dans le keystore — utilisé plus tard dans la config
`-keyalg RSA`	Algorithme de chiffrement standard Android
`-keysize 2048`	Taille de la clé en bits (minimum recommandé par Google)
`-validity 10000`	Durée de validité en jours (~27 ans). Google exige min. 25 ans
`-keystore ~/upload-keystore.jks`	Chemin de création du fichier keystore (racine de votre home)
`-storetype JKS`	Format Java KeyStore — standard Android

Le terminal va vous demander de saisir les informations suivantes :

1. Keystore password → choisissez un mot de passe, notez-le
2. Confirm password → répétez-le
3. First and last name → votre nom ou le nom de l'app
4. Organizational unit → laissez vide (appuyez sur Entrée)
5. Organization → ex : MetanoiaTek
6. City → ex : Paris
7. State → ex : IDF
8. Country code → FR
9. Confirmation → tapez yes
10. Key password → appuyez sur Entrée (même mot de passe que le keystore)

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

renelsmac@Mac-Renel saws_app % keytool -genkeypair \
  -alias upload \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -keystore ~/upload-keystore.jks \
  -storetype JKS
Enter keystore password:
Re-enter new password:
What is your first and last name?
  [Unknown]:  Renel Sainfort
What is the name of your organizational unit?
  [Unknown]:
What is the name of your organization?
  [Unknown]:  Metanoiatek
What is the name of your City or Locality?
  [Unknown]:  Paris
What is the name of your State or Province?
  [Unknown]:  IDF
What is the two-letter country code for this unit?
  [Unknown]:  FR
Is CN=Renel Sainfort, OU=Unknown, O=Metanoiatek, L=Paris, ST=IDF, C=FR correct?
  [no]:  yes

Generating 2 048 bit RSA key pair and self-signed certificate (SHA256withRSA) with a validity of 10 000 days
        for: CN=Renel Sainfort, OU=Unknown, O=Metanoiatek, L=Paris, ST=IDF, C=FR
Enter key password for <upload>
        (RETURN if same as keystore password):
```

> ⚠️ *Ce fichier est précieux et unique — si vous le perdez, vous ne pourrez plus mettre à jour votre app avec la même identité. Faites-en une sauvegarde.*

Vérifiez que le fichier a bien été créé :

```
ls -lh ~/upload-keystore.jks
```

```
renelsmac@Mac-Renel saws_app % ls -lh ~/upload-keystore.jks
-rw-r--r--@ 1 renelsmac  staff   2.2K Mar 16 14:07 /Users/renelsmac/upload-keystore.jks
renelsmac@Mac-Renel saws_app %
```

Étape 3 — Créer le fichier key.properties

Ce fichier stocke les informations de signature pour que Flutter sache comment signer l'APK.

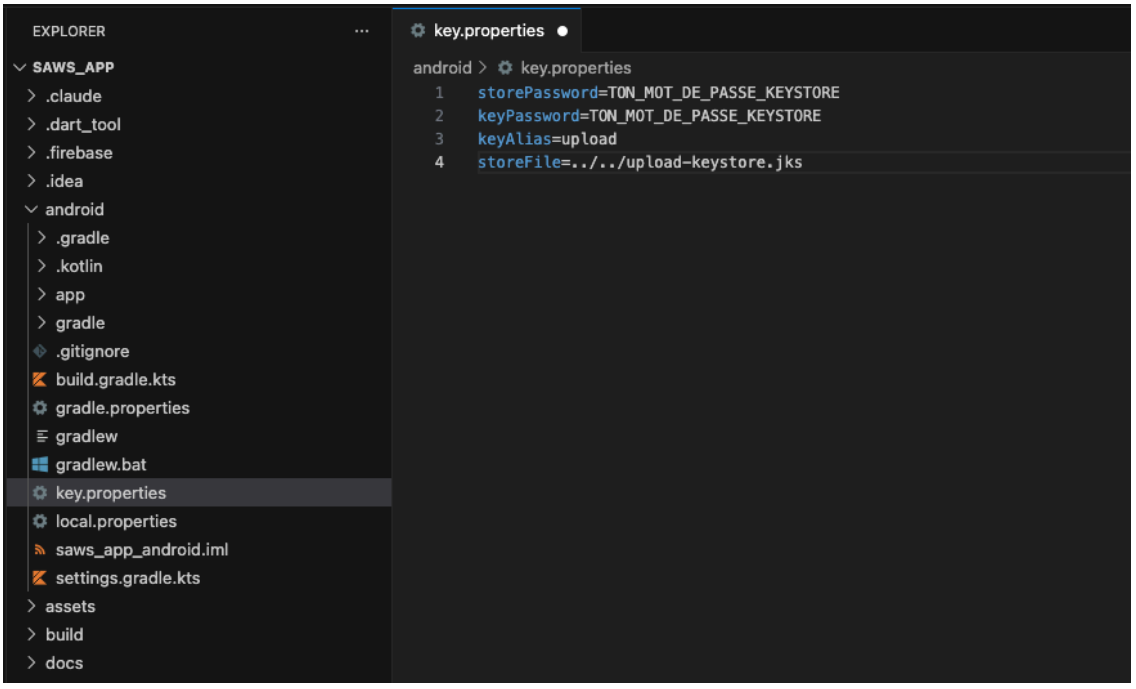
Dans votre projet Flutter, créez le fichier :

```
touch android/key.properties
```

Ouvrez-le dans VS Code et collez le contenu suivant (remplacez TON_MOT_DE_PASSE par votre mot de passe) :

```
storePassword=TON_MOT_DE_PASSE
keyPassword=TON_MOT_DE_PASSE
keyAlias=upload
storeFile=../../upload-keystore.jks
```

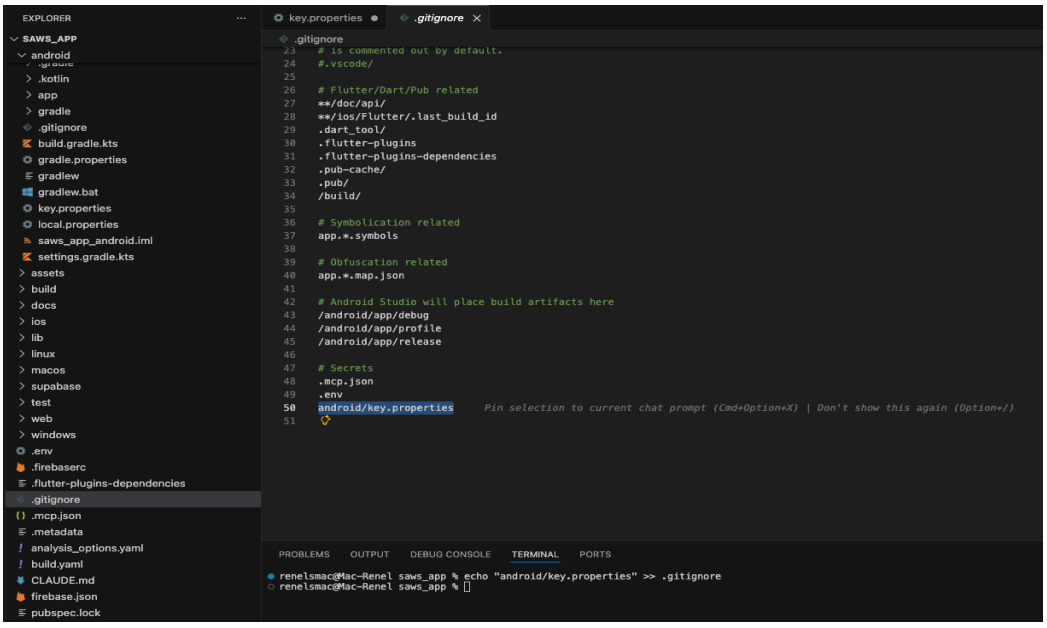
> ℹ️ *storeFile=../../ est le chemin relatif depuis android/app/ vers votre home ~/. Ne le modifiez pas.*



Ajoutez ce fichier au .gitignore pour ne jamais pusher vos mots de passe :

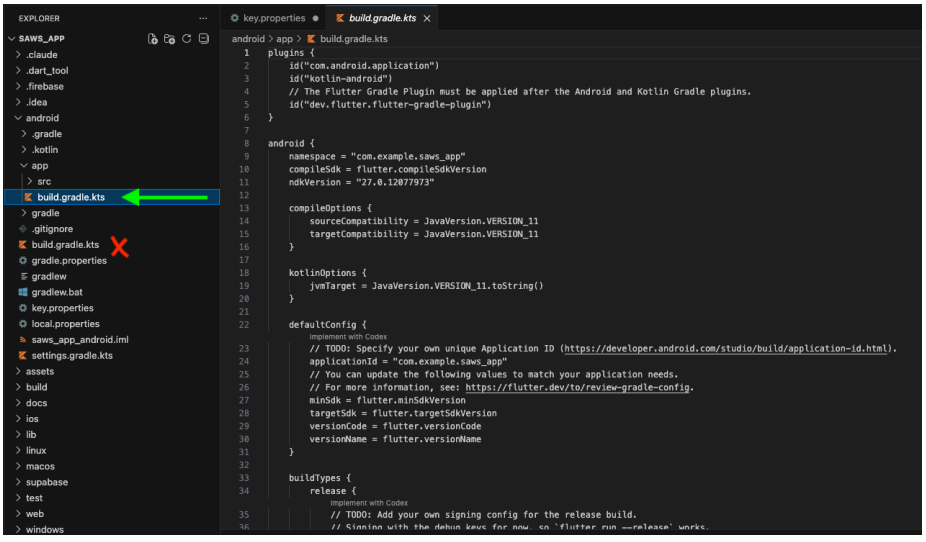
```
echo "android/key.properties" >> .gitignore
```

> ⚠️ *Ne committez JAMAIS key.properties sur GitHub — il contient vos mots de passe en clair.*



Étape 4 — Configurer build.gradle.kts

Ouvrez le fichier android/app/build.gradle.kts dans VS Code et effectuez 4 modifications :



1. Tout en haut — les imports

Ajoutez ces deux lignes tout en haut du fichier, avant le bloc plugins :

```
// ↓ IMPORTS — tout en haut du fichier, avant plugins
import java.util.Properties
import java.io.FileInputStream
```

2. Entre plugins {} et android {} — lecture du key.properties

```
// ↓ PLUGINS — bloc existant, ne pas toucher
plugins {
    ...
}

// ↓ AJOUTER ICI — lecture du key.properties
val keyPropertiesFile = rootProject.file("key.properties")
val keyProperties = Properties()
keyProperties.load(FileInputStream(keyPropertiesFile))
```

3. Dans android {} — signingConfigs et buildTypes

```
// ↓ ANDROID — bloc existant
android {

    // ↓ AJOUTER ICI — avant buildTypes
    signingConfigs {
        create("release") {
            keyAlias = keyProperties["keyAlias"] as String
            keyPassword = keyProperties["keyPassword"] as String
            storeFile = file(keyProperties["storeFile"] as String)
            storePassword = keyProperties["storePassword"] as String
        }
    }

    // ↓ MODIFIER — bloc existant
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = false
        }
    }
}
```

L'ordre final dans le fichier doit être :

```
imports            ← tout en haut
plugins { }
val keyProperties...
android {
    signingConfigs { }
    buildTypes { }
}
```

```
android > app > build.gradle.kts
1
2   //↓ IMPORTS — tout en haut du fichier, avant plugins
3   import java.util.Properties
4   import java.io.FileInputStream
5
6
7
8   plugins {
9       id("com.android.application")
10      id("kotlin-android")
11      // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
12      id("dev.flutter.flutter-gradle-plugin")
13  }
14
15  // ↓ AJOUTER ICI — lecture du key.properties
16  val keyPropertiesFile = rootProject.file("key.properties")
17  val keyProperties = Properties()
18  keyProperties.load(FileInputStream(keyPropertiesFile))     Pin selection to current chat prompt (Cmd+Option+X) | Don't show this again (Option+
19
20  android {
21      namespace = "com.example.saws_app"
22      compileSdk = flutter.compileSdkVersion
23      ndkVersion = "27.0.12077973"
24
25      compileOptions {
26          sourceCompatibility = JavaVersion.VERSION_11
27          targetCompatibility = JavaVersion.VERSION_11
28      }
29
30      kotlinOptions {
31          jvmTarget = JavaVersion.VERSION_11.toString()
32      }
33
34      defaultConfig {
            Implement with Codex
35          // TODO: Specify your own unique Application ID (https://developer.android.com/studio/build/application-id.html).
36          applicationId = "com.example.saws_app"
```

```
33      }
34      defaultConfig {
            Implement with Codex
35          // TODO: Specify your own unique Application ID (https://developer.android.com/studio/build/application-id.html).
36          applicationId = "com.example.saws_app"
37          // You can update the following values to match your application needs.
38          // For more information, see: https://flutter.dev/to/review-gradle-config.
39          minSdk = flutter.minSdkVersion
40          targetSdk = flutter.targetSdkVersion
41          versionCode = flutter.versionCode
42          versionName = flutter.versionName
43      }
44
45      // ↓ AJOUTER ICI — avant buildTypes
46      signingConfigs {
47          create("release") {
48              keyAlias = keyProperties["keyAlias"] as String
49              keyPassword = keyProperties["keyPassword"] as String
50              storeFile = file(keyProperties["storeFile"] as String)
51              storePassword = keyProperties["storePassword"] as String
52          }
53      }     Pin selection to current chat prompt (Cmd+Option+X) | Don't show this again (Option+/)
54
55      buildTypes {
56          release {
57              ↓ MODIFIER — bloc existant
58              signingConfig = signingConfigs.getByName("release")
59              isMinifyEnabled = false
60          }
61      }
62  }
63
64  flutter {
65      source = "../.."
66  }
```

Étape 5 — Builder l'APK signé

Dans votre terminal, assurez-vous d'être à la racine de votre projet Flutter puis lancez :

```
flutter build apk --release
```

Le build peut prendre 1 à 3 minutes selon votre machine.

> ℹ️ *Les warnings source value 8 is obsolete sont normaux — ils n'empêchent pas le build de fonctionner.*

À la fin, vous devez voir :

```
✓  Built build/app/outputs/flutter-apk/app-release.apk
```

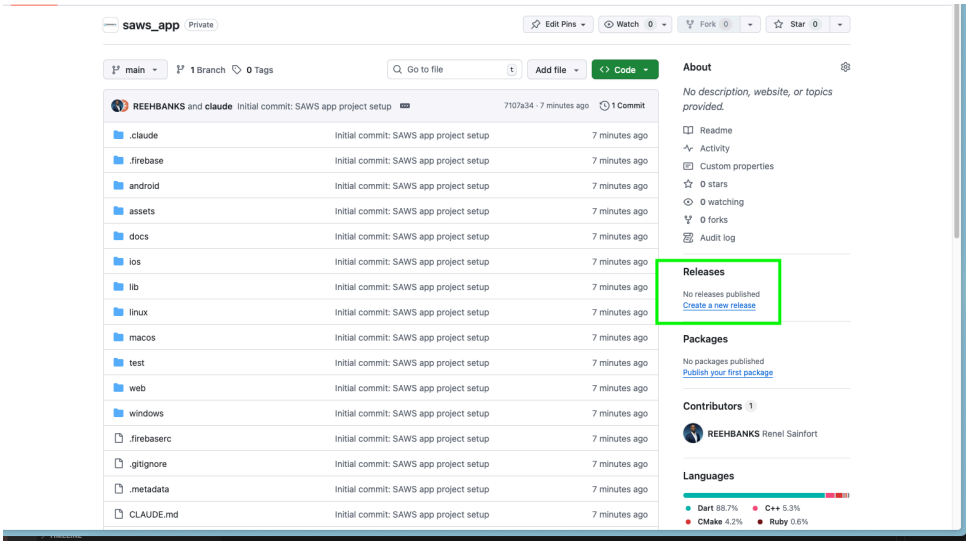
```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

renelsmac@Mac-Renel saws_app % flutter build apk --release

  A new version of Flutter is available!

  To update to the latest version, run "flutter upgrade".

Font asset "CupertinoIcons.ttf" was tree-shaken, reducing it from 257628 to 848 bytes (99.7% reduction). Tree-shaking can be disabled by providing the --n
o-tree-shake-icons flag when building your app.
Font asset "MaterialIcons-Regular.otf" was tree-shaken, reducing it from 1645184 to 11388 bytes (99.3% reduction). Tree-shaking can be disabled by providi
ng the --no-tree-shake-icons flag when building your app.
warning: [options] source value 8 is obsolete and will be removed in a future release
warning: [options] target value 8 is obsolete and will be removed in a future release
warning: [options] To suppress warnings about obsolete options, use -Xlint:-options.
3 warnings
Running Gradle task 'assembleRelease'...                          463,3s
✓ Built build/app/outputs/flutter-apk/app-release.apk (85.9MB)
renelsmac@Mac-Renel saws_app %
```

Votre APK signé se trouve dans :

```
VOTRE_PROJET/build/app/outputs/flutter-apk/app-release.apk
```

Étape 6 — Créer la Release GitHub

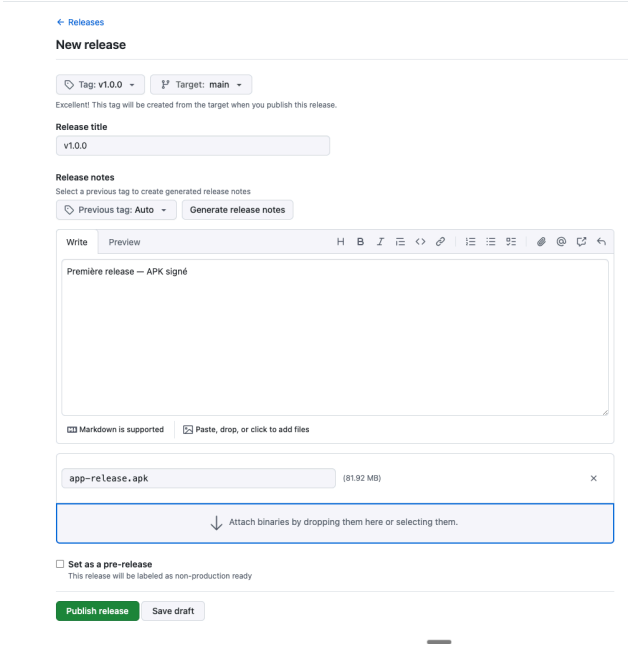
Ouvrez votre repo sur github.com. Dans le panneau de droite, cliquez sur Releases puis Create a new release.



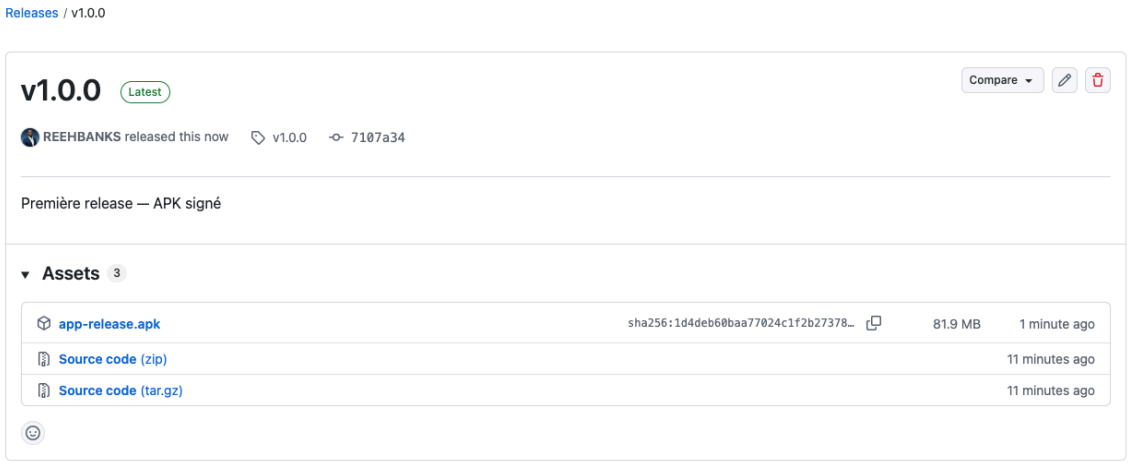
Remplissez le formulaire :

11. **Choose a tag** → tapez `v1.0.0` puis cliquez sur *Create new tag: v1.0.0*
12. **Release title** → tapez `v1.0.0`
13. **Description** → optionnel, ex : *Première release — APK signé*
14. **Attach binaries** → glissez-déposez votre `app-release.apk` depuis le Finder

> 💡 *Renommez l'APK avant de l'uploader pour l'identifier facilement, ex : nomapp-v1.0.0.apk*



Cliquez sur le bouton vert Publish release.



Pour aller plus loin — APK vs AAB

Ce tutoriel génère un APK (Android Package) — un fichier installable directement sur un appareil Android, sans passer par le Play Store. C'est le format idéal pour distribuer votre app à un testeur ou un correcteur.

Il existe un second format : l'AAB (Android App Bundle). C'est le format exigé par le Google Play Store. Contrairement à l'APK, l'AAB ne s'installe pas directement — il est envoyé à Google qui se charge d'optimiser et de distribuer l'app selon les appareils.

	APK	AAB
Distribution	`Direct (side-loading)`	`Google Play Store uniquement`
Installation	`Directement sur l'appareil`	`Via le Play Store`
Commande Flutter	`flutter build apk --release`	`flutter build appbundle --release`
Fichier généré	`build/app/outputs/flutter-apk/app-release.apk`	`build/app/outputs/bundle/release/app-release.aab`
Usage	`Tests, remise de projet, partage direct`	`Publication sur le Play Store`

> 💡 *La procédure de signature est identique pour les deux formats. Seule la commande de build change.*

Récapitulatif

✓ 1	Vérifier keytool
✓ 2	Créer la Keystore (upload-keystore.jks)
✓ 3	Créer android/key.properties + l'ajouter au .gitignore
✓ 4	Configurer android/app/build.gradle.kts
✓ 5	flutter build apk --release
✓ 6	Créer la Release GitHub avec l'APK en asset